```
    distance, score = evaluate_fitness(station)
    print(f"📌 Station {station['id']}: Distance = {distance:.2f} km | Traffic Risk Score = {score:.2f}")

for b_station in BUSY_STATIONS:
    print(f"❌ [Skipped] Station {b_station['id']} is currently BUSY.")

optimal_station = AVAILABLE_STATIONS[gbest_position]
final_distance, final_score = evaluate_fitness(optimal_station)

print("\n" + "="*60)
print(f"🟢 OPTIMAL STATION DISPATCHED: {optimal_station['id']}")
print(f"📍 Station Location: ({optimal_station['lat']}, {optimal_station['lon']})")
print(f"🧮 Traffic-Adjusted Route Score: {final_score:.2f} (Lowest overall risk)")
print("="*60)


output_dictionary = {
    'id': optimal_station['id'],
    'lat': optimal_station['lat'],
    'lon': optimal_station['lon'],
    'status': optimal_station['status']
}
print(output_dictionary)
```

```
🚨 EMERGENCY ALERT: Accident reported at (12.948, 77.712) in East_Bangalore
Scanning regional network for optimal dispatch units...

📌 Station STN_WHITEFIELD: Distance = 4.77 km | Traffic Risk Score = 11.93
📌 Station STN_MARATHAHALLI: Distance = 1.82 km | Traffic Risk Score = 4.56
❌ [Skipped] Station STN_INDIRANAGAR is currently BUSY.

============================================================
🟢 OPTIMAL STATION DISPATCHED: STN_MARATHAHALLI
📍 Station Location: (12.9562, 77.6974)
🧮 Traffic-Adjusted Route Score: 4.56 (Lowest overall risk)
============================================================
{'id': 'STN_MARATHAHALLI', 'lat': 12.9562, 'lon': 77.6974, 'status': 'Available'}
```

[6]

```
gbest_idx = pbest_fitnesses.index(min(pbest_fitnesses))
gbest_position = pbest_positions[gbest_idx]

w, c1, c2 = 0.5, 1.0, 1.0

for t in range(iterations):
    for i in range(swarm_size):
        r1, r2 = random.random(), random.random()


        particle_velocities[i] = (w * particle_velocities[i] +
                                  c1 * r1 * (pbest_positions[i] - particle_positions[i]) +
                                  c2 * r2 * (gbest_position - particle_positions[i]))


        new_position = int(round(particle_positions[i] + particle_velocities[i]))


        if new_position < 0: new_position = 0
        if new_position >= num_stations: new_position = num_stations - 1
        particle_positions[i] = new_position


        _, current_fitness = evaluate_fitness(AVAILABLE_STATIONS[particle_positions[i]])


        if current_fitness < pbest_fitnesses[i]:
            pbest_fitnesses[i] = current_fitness
            pbest_positions[i] = particle_positions[i]


            _, current_gbest_fitness = evaluate_fitness(AVAILABLE_STATIONS[gbest_position])
            if current_fitness < current_gbest_fitness:
                gbest_position = particle_positions[i]


for station in AVAILABLE_STATIONS:
    distance, score = evaluate_fitness(station)
    print(f"📌 Station {station['id']}: Distance = {distance:.2f} km | Traffic Risk Score = {score:.2f}")
```

```python
import math
import random

AVAILABLE_STATIONS = [
    {"id": "STN_WHITEFIELD", "lat": 12.9698, "lon": 77.7500, "status": "Available"},
    {"id": "STN_MARATHAHALLI", "lat": 12.9562, "lon": 77.6974, "status": "Available"}
]

BUSY_STATIONS = [
    {"id": "STN_INDIRANAGAR", "lat": 12.9719, "lon": 77.6412, "status": "Busy"}
]

accident_lat = 12.948
accident_lon = 77.712
traffic_multiplier = 2.5

print(f"🚨 EMERGENCY ALERT: Accident reported at ({accident_lat}, {accident_lon}) in East_Bangalore")
print("Scanning regional network for optimal dispatch units...\n")

def evaluate_fitness(station):
    d_lat = (station["lat"] - accident_lat) * 111.0
    d_lon = (station["lon"] - accident_lon) * 111.0 * math.cos(math.radians(accident_lat))
    distance_km = math.sqrt(d_lat**2 + d_lon**2)

    fitness = distance_km * traffic_multiplier
    return distance_km, fitness

num_stations = len(AVAILABLE_STATIONS)
swarm_size = 6
iterations = 5


particle_positions = [random.randint(0, num_stations - 1) for _ in range(swarm_size)]
particle_velocities = [0.0 for _ in range(swarm_size)]

pbest_positions = list(particle_positions)
pbest_fitnesses = []
for pos in pbest_positions:
    _, score = evaluate_fitness(AVAILABLE_STATIONS[pos])
    pbest_fitnesses.append(score)
```